



**CPE9: OPERATING SYSTEMS
LABORATORY ACTIVITY 4**

Name: Jun Michael C. Salvador
Section: BSCPE 2B

Date: April 5, 2026

System Monitoring and Bash Scripting via WSL

I. Objective

By the end of this laboratory activity, participants will be able to:

- a. Install and configure the Windows Subsystem for Linux (WSL) on a Windows host.
- b. Understand the fundamental structure and syntax of Bash shell scripts.
- c. Analyze a system monitoring script to understand how Linux commands extract system information.
- d. Create, modify, set permissions for, and execute shell scripts within a Linux environment.

II. Methodology and Setup

- a. A computer running Windows 10 (Version 2004 or higher) or Windows 11.
- b. An active internet connection to download the Linux distribution.
- c. Administrator privileges on the Windows machine.

Setting up the Windows Subsystem for Linux:

WSL allows you to run a full Linux environment directly on Windows without the overhead of a traditional virtual machine or dual-boot setup.

1. Open PowerShell as Administrator:

- a. Right-click the Windows Start button.
- b. Select Windows Terminal (Run as Admin) or Windows PowerShell (Run as Admin).
- c. If prompted by User Account Control (UAC), click Yes.

2. Install WSL:

- a. Type the following command and press Enter : `wsl --install`
Note: This command enables the necessary Windows features and installs Ubuntu (the default Linux distribution) by default.

3. Restart the Computer:

- a. Once the installation process finishes, reboot your PC to apply the changes.

4. Initialize the Linux Environment:

- a. After rebooting, open the Start menu, type Ubuntu, and open the application.
- b. A console window will open, and it may take a few minutes to decompress and initialize files for the first time.
- c. If there is no Ubuntu, open your Windows Powershell and type: `wsl --list --verbose`
 - i. If it says "Windows Subsystem for Linux has no installed distributions": This means that Ubuntu was never downloaded. To download Ubuntu, proceed to 'd'.
- d. In your PowerShell window, type the following command and press Enter: `wsl --install -d Ubuntu`
 - i. You should see a progress bar indicating that it is downloading.
 - ii. Once the download finishes, one of two things will happen:
 1. **Automatic:** A new window should automatically pop up saying "Installing, this may take a few minutes..." followed by a prompt to create your UNIX username and password.
 2. **Manual:** If a new window doesn't pop up, simply type ubuntu into your PowerShell window and press Enter to kickstart the initialization.

5. Set Up User Credentials:

- a. When prompted, create a UNIX username. (This does not need to match your Windows username).
- b. Enter a password. Note: For security reasons, characters will not appear on the screen as you type. Type carefully and press Enter.
- c. Confirm your password.

III. Introduction to Bash Scripting

A shell script is simply a text file containing a sequence of commands for a UNIX-based operating system to execute. Writing scripts allows administrators to automate repetitive tasks, manage systems, and process data efficiently.

Core Components of a Bash Script:

- **The Shebang (`#!/bin/bash`):** Always placed on the very first line. It tells the operating system which interpreter to use to run the script. In this case, it specifies the Bash shell.

- **Variables:** Used to store data. In Bash, you define a variable without spaces around the equals sign (e.g., name="Linux"). You call the variable using the \$ symbol (e.g., echo \$name).
- **Command Substitution (\$(command)):** Allows the output of a command to replace the command itself. For example, current_date=\$(date) saves the output of the date command into the variable.
- **Functions:** Blocks of reusable code. They help keep scripts organized and modular.

Bash

```
function my_function() {
    echo "This is inside a function."
}
```

How to Write Various Scripts:

When writing your own scripts, follow a standard workflow:

1. **Define the Goal:** What manual task are you trying to automate? (e.g., backing up a folder, checking disk space).
2. **Test Commands Interactively:** Run the commands one by one in the terminal first to ensure they work as expected.
3. **Draft the Script:** Use a terminal text editor (like nano or vim) to assemble the commands, add variables for flexibility, and group logic into functions.
4. **Add Comments:** Use the # symbol to document what each part of your script does.
5. **Test and Refine:** Run the script in a safe environment and fix any errors.

IV. Procedures and Task

Task 1: Create the Script File

1. Open your Ubuntu terminal.
2. Ensure you are in your home directory by typing:
`cd ~`
3. Open the nano text editor and create a new file named monitoring.sh:
`nano monitoring.sh`

Task 2: Input the Code

1. Copy the entirety of the provided Bash script:

```
#!/bin/bash
##
# BASH script that checks:
# - Memory usage
# - CPU load
# - Number of TCP connections
# - Kernel version
##
server_name=$(hostname)

function memory_check() {
    echo -e "\nMemory usage on ${server_name} is: "
    free -h
}

function cpu_check() {
    echo -e "\nCPU load on ${server_name} is: "
    uptime
}

function tcp_check() {
    echo -e "\nTCP connections on ${server_name}: "
    cat /proc/net/tcp | wc -l
}

function kernel_check() {
    echo -e "\nKernel version on ${server_name} is: "
    uname -r
}

function all_checks() {
```

```
memory_check
cpu_check
tcp_check
kernel_check
}

all_checks
```

2. Paste it into the nano editor. (*To paste in the WSL terminal, simply right-click your mouse*).
3. Save the file by pressing Ctrl + O (the letter O), then press Enter.
4. Exit the editor by pressing Ctrl + X.

Task 3: Grant Execution Permissions

By default, Linux creates text files without permission to be run as programs to prevent malicious software from executing. You must manually grant "execute" permissions.

1. Run the chmod (change mode) command:
chmod +x monitoring.sh
2. *Optional:* Verify the permissions changed by typing ls -l. You should see an x in the file's permission string, and the filename will appear green.

Task 4: Run the Script

1. Execute the script by calling it with ./ (which tells the system to look in the current directory):
./monitoring.sh
2. **Observe the Output:** Review the data printed to your terminal. Take note of your system's memory usage, uptime, active TCP connections, and kernel version.

After executing ./monitoring.sh, carefully review the output printed in your terminal. Use the data generated by the script to answer the following questions:

1. **System Identification:** What is the server_name (hostname) displayed in your output?.

Answer: My hostname is **LAPTOP-FGF88A6L**, which is shown in every output line of the script.

2. **Memory Metrics (free -h):** * What is the total amount of memory available on your subsystem?
 - o How much memory is currently being used, and how much is free?

Answer: The total memory is 7.6Gi. Currently, 516Mi is being used and 6.6Gi is free.

3. **CPU and Uptime (uptime):**
 - o How long has your WSL environment been running?
 - o What are the three system load averages shown at the end of the line? (*Note: These represent the load over the last 1, 5, and 15 minutes*).

Answer: My WSL has been running for **6 minutes**. The load averages are **0.00, 0.01, and 0.00** for the last 1, 5, and 15 minutes.

4. **Network Activity (cat /proc/net/tcp | wc -l):** How many active TCP connections are currently established on your system?

Answer: There are 4 active TCP connections currently on my system.

5. **System Information (uname -r):** What specific Linux kernel version is your WSL environment currently running?

Answer: The kernel version is 6.6.87.2-microsoft-standard-WSL2.

V. Observation and Results

Based on your execution of the script and your understanding of the Linux command line, answer the following discussion questions:

1. Explain the purpose of the `#!/bin/bash` line at the very beginning of the script. What would happen if this line were omitted or changed?

Answer: The `#!/bin/bash` line tells the system to use Bash to run the script. If it's removed, the script might not run properly or could throw an error since the system won't know which interpreter to use.

2. The script uses the syntax `server_name=$(hostname)`. Explain what the `$()` syntax does and why it is useful when writing automation scripts.

Answer: The `$()` syntax runs a command and saves its output into a variable. It is useful because it allows the script to automatically get and use system information.

3. Why is it critical for system administrators to monitor metrics like CPU load and memory usage? What performance issues might occur if the system runs out of free memory?

Answer: Monitoring CPU and memory helps make sure the system isn't overloaded. If memory runs out, the system will slow down significantly and may start crashing or freezing since it has nowhere to store active processes.

4. In the `tcp_check` function, the command `cat /proc/net/tcp | wc -l` is used. Describe the role of the pipe operator (`|`) in this specific command sequence.

Answer: The pipe `|` sends the output of one command to another command. In this case, it passes the TCP data to `wc -l` to count the number of lines.

5. If you wanted to modify this script to also check the available disk space on your primary hard drive, which Linux command would you use, and how would you structure the new function?

Answer: I would use the `df -h` command to check disk space. I also would create a new function and place the command inside it similar to the other checks.

VI. Conclusion

(Write a concise paragraph summarizing what you learned about pre-emptive scheduling and time-slicing from this simulation.)

Through this activity, I learned how to set up WSL and write a basic Bash script that monitors system information. It showed me how useful scripting is for automating tasks that would otherwise require running multiple commands manually. I also got a better understanding of how Linux tracks system resources like memory, CPU load, network connections, and kernel version, which are all important things to keep an eye on when managing a system.